

ui.refreshable 全面解析：NiceGUI 可刷新 UI 核心工具

`ui.refreshable` 是 NiceGUI 框架中用于创建**可动态刷新 UI 组件**的核心装饰器，其核心能力是为函数或方法添加 `refresh()` 方法，调用该方法时会自动删除函数 / 方法创建的所有 UI 元素并重新生成，从而实现 UI 状态的实时更新。适用于动态数据展示、输入验证、多状态切换等场景，同时支持全局 / 局部作用域、异步操作、响应式状态等高级特性。

一、核心基础：装饰器与核心方法

1. 两个核心装饰器

装饰器	适用场景	说明
<code>@ui.refreshable</code>	全局函数、局部函数（如页面函数内）	为普通函数添加刷新能力
<code>@ui.refreshable_method</code>	类中的方法	功能与 <code>@ui.refreshable</code> 一致，避免静态类型检查错误

2. 关键方法

(1) `refresh(*args: Any, **kwargs) -> AwaitableResponse`

- 核心功能：刷新由装饰函数创建的 UI 元素（删除旧元素 + 重建新元素）。
- 参数特性：支持传递与原函数相同的参数或子集，会自动与原函数参数合并（实现“带参数刷新”）。
- 异步支持：若装饰的是异步函数（`async def`），可通过 `await refresh()` 等待刷新完成；同步函数调用时，刷新操作在后台异步执行。

(2) `prune() -> None`

- 功能：自动清理不再处于客户端连接页面的目标元素（避免内存泄漏）。
- 调用时机：每次执行 `refresh()` 前会自动调用，无需手动触发。

二、核心使用场景与示例

1. 基础可刷新 UI（无参数）

适用于简单的动态数据展示，通过外部操作触发 UI 刷新。

```
import random
from nicegui import ui

numbers = [] # 全局数据存储

@ui.refreshable # 装饰可刷新函数
def number_ui() -> None:
```

```
# 展示排序后的数字列表
ui.label(', '.join(str(n) for n in sorted(numbers)))

def add_number() -> None:
    numbers.append(random.randint(0, 100))
    number_ui.refresh() # 触发 UI 刷新

number_ui() # 初始渲染
ui.button('Add random number', on_click=add_number) # 点击添加数字并刷新
ui.run()
```

- 逻辑：点击按钮时，`numbers` 列表添加随机数，调用 `number_ui.refresh()` 重建标签，展示最新排序结果。

2. 带参数的可刷新 UI

支持在刷新时传递不同参数，实现“同一 UI 组件切换不同状态”（如多时区时钟）。

```
import pytz
from datetime import datetime
from nicegui import ui

@ui.refreshable # 装饰带参数的函数
def clock_ui(timezone: str):
    ui.label(f'Current time in {timezone}:')
    # 根据传入的时区参数展示当前时间
    ui.label(datetime.now(tz=pytz.timezone(timezone)).strftime('%H:%M:%S'))

# 初始渲染（默认柏林时区）
clock_ui('Europe/Berlin')
# 刷新按钮：传递不同时区参数
ui.button('Refresh Berlin', on_click=clock_ui.refresh) # 无参数时复用原参数
ui.button('Refresh New York', on_click=lambda: clock_ui.refresh('America/New_York'))
ui.button('Refresh Tokyo', on_click=lambda: clock_ui.refresh('Asia/Tokyo'))
ui.run()
```

- 关键：`refresh()` 可接收与原函数匹配的参数，实现“参数化刷新”，无需重新定义 UI 结构。

3. 输入验证反馈（实时刷新）

结合输入组件的 `on_change` 事件，实现输入合法性的实时反馈（如密码强度校验）。

```
import re
from nicegui import ui

# 密码输入框：输入变化时触发刷新
pwd = ui.input('Password', password=True, on_change=lambda: show_info.refresh())

# 密码校验规则（键：规则描述，值：校验函数）
rules = {
    'Lowercase letter': lambda s: re.search(r'[a-z]', s),
    'Uppercase letter': lambda s: re.search(r'[A-Z]', s),
}
```

```

'Digit': lambda s: re.search(r'\d', s),
'Special character': lambda s: re.search(r"[!@#$$%^&*(),.?':{}|<>]", s),
'min. 8 characters': lambda s: len(s) >= 8,
}

@ui.refreshable
def show_info():
    for rule, check in rules.items():
        with ui.row().classes('items-center gap-2'):
            # 根据校验结果展示不同图标和文字样式
            if check(pwd.value or ''):
                ui.icon('done', color='green')
                ui.label(rule).classes('text-xs text-green strike-through') # 已满足（绿色划线）
            else:
                ui.icon('radio_button_unchecked', color='red')
                ui.label(rule).classes('text-xs text-red') # 未满足（红色）

show_info() # 初始渲染
ui.run()

```

- 逻辑：输入框内容变化时，调用 `show_info.refresh()` 重新校验所有规则并更新 UI 反馈。

4. 响应式状态（`ui.state` 结合刷新）

通过 `ui.state` 创建响应式变量，变量更新时自动触发 UI 刷新，无需手动调用 `refresh()`。

```

from nicegui import ui

@ui.refreshable
def counter(name: str):
    with ui.card():
        # 响应式变量：count（计数）、color（文字颜色）
        count, set_count = ui.state(0)
        color, set_color = ui.state('black')

        # 展示响应式变量（变量更新时自动刷新）
        ui.label(f'{name} = {count}').classes(f'text-{color}')
        # 按钮：更新 count 变量（自动触发 UI 刷新）
        ui.button(f'{name} += 1', on_click=lambda: set_count(count + 1))
        # 下拉框：更新 color 变量（自动触发 UI 刷新）
        ui.select(['black', 'red', 'green', 'blue'],
                  value=color, on_change=lambda e: set_color(e.value))

# 同时创建两个独立计数器
with ui.row():
    counter('A')
    counter('B')

ui.run()

```

- 核心：`ui.state` 返回“变量值 + 变量更新函数”，调用更新函数时，依赖该变量的 UI 元素会自动刷新，无需显式调用 `refresh()`。

5. 异步可刷新 UI（可等待刷新）

支持装饰异步函数，通过 `await refresh()` 等待刷新完成，适用于需要协调 UI 状态的场景（如禁用按钮防止重复点击）。

```
import asyncio
from uuid import uuid4
from nicegui import events, ui

@ui.refreshable # 装饰异步函数
async def compute():
    await asyncio.sleep(1) # 模拟耗时操作（如接口请求）
    ui.label(str(uuid4())) # 生成随机 UUID 展示

async def handle_click(e: events.ClickEventArguments):
    e.sender.disable() # 点击后禁用按钮
    await compute.refresh() # 等待刷新完成（确保耗时操作结束）
    e.sender.enable() # 刷新完成后启用按钮

async def root():
    ui.button('Refresh', on_click=handle_click)
    await compute() # 初始渲染异步函数

ui.run(root)
```

- 关键：异步函数的 `refresh()` 是可等待的，可用于控制 UI 交互时序（如避免重复触发耗时操作）。

三、作用域控制：全局 vs 局部

`ui.refreshable` 的作用域决定了多个 UI 实例是否共享状态、是否同步刷新，核心分为**全局作用域**和**局部作用域**。

1. 全局作用域

- 定义：可刷新函数在**全局范围**（页面函数外）定义。
- 特性：所有通过该函数创建的 UI 实例共享状态，刷新时所有实例同步更新（跨标签页、跨实例）。
- 示例：

```
from datetime import datetime
from nicegui import ui

# 全局可刷新函数
@ui.refreshable
def time():
    ui.label(f'Time: {datetime.now()}')

# 页面 1: 展示时间
@ui.page('/global_refreshable')
def demo():
    time()
    ui.button('Refresh', on_click=time.refresh) # 点击后所有实例同步刷新
```

```
# 首页：跳转链接
@ui.page('/')
def page():
    ui.link('Open demo', demo)

ui.run()
```

- 效果：打开多个标签页访问 `/global_refreshable`，点击任意标签页的“Refresh”，所有标签页的时间会同步更新。

2. 局部作用域（3 种实现方式）

局部作用域的核心是：每个 UI 实例拥有独立状态，刷新时仅当前实例更新（不影响其他实例）。

（1）局部作用域 A：页面函数内定义可刷新函数

将可刷新函数定义在**页面函数内部**，每个页面实例会创建独立的函数副本，状态隔离。

```
from datetime import datetime
from nicegui import ui

@ui.page('/local_refreshable_a')
def demo():
    # 页面内局部可刷新函数
    @ui.refreshable
    def time():
        ui.label(f'Time: {datetime.now()}')

    time()
    ui.button('Refresh', on_click=time.refresh) # 仅当前页面实例刷新

@ui.page('/')
def page():
    ui.link('Open demo', demo)

ui.run()
```

- 效果：多个标签页访问 `/local_refreshable_a`，点击某个标签页的“Refresh”，仅该标签页的时间更新。

（2）局部作用域 B：类的可刷新方法（`@ui.refreshable_method`）

通过类封装可刷新方法，每个类实例拥有独立状态，适用于需要创建多个独立组件的场景（如多个独立时钟）。

```
from datetime import datetime
from nicegui import ui

class Clock:
    # 类方法使用 @ui.refreshable_method 装饰
    @ui.refreshable_method
    def time(self):
        ui.label(f'Time: {datetime.now()}')
```

```

@ui.page('/local_refreshable_b')
def demo():
    # 创建两个独立的 clock 实例
    clock1 = clock()
    clock2 = clock()

    clock1.time()
    ui.button('Refresh Clock 1', on_click=clock1.time.refresh) # 仅刷新 clock1

    clock2.time()
    ui.button('Refresh Clock 2', on_click=clock2.time.refresh) # 仅刷新 clock2

@ui.page('/')
def page():
    ui.link('Open demo', demo)

ui.run()

```

- 核心: `@ui.refreshable_method` 作用于类实例, 而非类本身, 因此每个实例的刷新操作相互独立。

(3) 局部作用域 C: 全局函数 + 页面内动态装饰

全局定义普通 UI 函数, 在页面函数内通过 `ui.refreshable(函数名)` 动态创建可刷新实例, 实现状态隔离。

```

from datetime import datetime
from nicegui import ui

# 全局普通函数 (无装饰器)
def time():
    ui.label(f'Time: {datetime.now()}')

@ui.page('/local_refreshable_c')
def demo():
    # 页面内动态装饰为可刷新函数
    refreshable_time = ui.refreshable(time)
    refreshable_time()
    ui.button('Refresh', on_click=refreshable_time.refresh) # 仅当前实例刷新

@ui.page('/')
def page():
    ui.link('Open demo', demo)

ui.run()

```

- 灵活度: 无需修改全局函数, 可在不同页面中动态创建独立的可刷新实例。

四、关键特性总结

1. **自动重建**: `refresh()` 会自动删除旧 UI 元素并重建, 无需手动清理。
2. **参数兼容**: `refresh()` 可接收原函数的参数, 支持参数化刷新。
3. **响应式集成**: 与 `ui.state` 无缝配合, 变量更新时自动刷新 UI。

4. **异步支持**: 异步函数的 `refresh()` 可等待, 便于控制交互时序。
5. **作用域灵活**: 全局作用域实现同步刷新, 局部作用域实现独立刷新, 满足不同场景需求。
6. **自动清理**: `prune()` 方法自动清理无效元素, 避免内存泄漏。

五、适用场景汇总

- 动态数据展示 (如实时更新的列表、统计数据)。
- 输入验证反馈 (如密码强度、表单合法性校验)。
- 多状态切换 (如多时区时钟、多视图切换)。
- 异步操作反馈 (如耗时请求的加载状态、结果展示)。
- 多实例独立组件 (如多个独立计数器、时钟)。

通过 `ui.refreshable`, NiceGUI 实现了“低代码、高灵活”的 UI 动态更新能力, 无需手动操作 DOM, 即可快速构建响应式、可交互的 web 界面。